

Structured Logging: Standards That Stick at Scale



Published on May 15, 2022



Webstack
Builders

Table of Contents

Why Structured Logging Matters	3
The Inconsistency Tax	3
Quantifying the Cost	4
Designing Your Log Schema	5
Schema Design Principles	5
Adopting Elastic Common Schema (ECS)	6
Custom Fields and Extensions	8
Correlation ID Implementation	9
Correlation ID Types	10
How Correlation Flows Through Services	12
Propagation Mechanisms	13
HTTP Header Propagation	13
Message Queue Correlation	16
Correlation Anti-Patterns	17
Logger Implementation Patterns	18
Type-Safe Logger Factory	18
Request-Scoped Logging	20
Sensitive Data Handling	22
Redaction Strategies	22
Field-Level Encryption	23
Collector-Side Processing	25
Log Pipeline Architecture	25
Noise Reduction Strategies	25
Schema Governance and Evolution	27
Schema Registry Options	27
Schema Evolution Process	28
Testing and Validating Your Logging	29
Schema Validation in CI	30
Correlation ID Propagation Tests	31
Log Quality Metrics	32
Conclusion	33



At 3 AM, I'm grepping through logs from twelve services trying to trace a single user request. One service logs `user_id=12345` , another logs `{"user": "12345"}` , a third logs `User 12345 logged in` . Same user, three different patterns. I write five regex queries, hit enter, and watch results scroll by – but something's wrong. The inventory service is silent. After twenty minutes, I discover it uses `customer_id` instead of `user_id` . My queries missed it entirely.

That's when I understood: logging JSON (JavaScript Object Notation) isn't the same as having a log *schema*.

Why Structured Logging Matters

The transition from `printf` -style logging to JSON (JavaScript Object Notation) is just the first step. Without a shared schema, you've replaced one problem (unstructured text) with another (incompatible structures). What matters isn't the format – it's the contract.

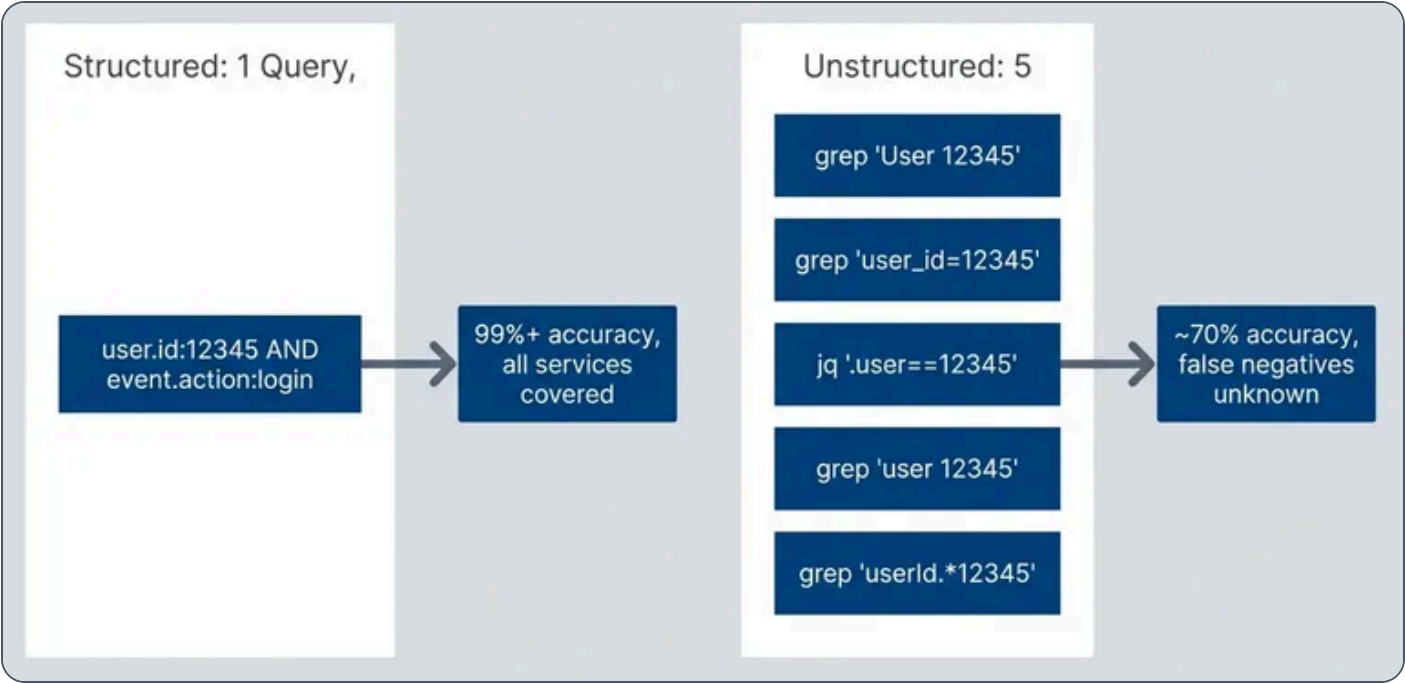
The Inconsistency Tax

Consider what happens when five services each log authentication events their own way:

Service A: User 12345 logged in from 192.168.1.1	
Service B: [INFO] Login successful - user_id=12345, ip=192.168.1.1	
Service C: { "user": "12345", "action": "login", "source_ip": "192.168.1.1" }	
Service D: 2024-01-15 14:30:00 INFO user 12345 authenticated	
Service E: { "userId": "12345", "event": "AUTH_SUCCESS" }	



Finding all logins for user 12345 now requires five different search patterns. Worse, you *don't know what you're missing*. Did you account for the service that uses `customer` instead of `user` ? The one that logs to a different index? The one with the typo in the field name?



Schema query coverage comparison.

Quantifying the Cost

The difference between ad-hoc logging and schema-based logging shows up in every metric that matters during incidents.

Metric	Ad-hoc Logging	Schema-based Logging	Impact
Time to find relevant logs	15 min	2 min	87% reduction
Query accuracy	~70%	99%+	30% fewer false negatives
Storage cost	Baseline	0.7x baseline	30% reduction



Metric	Ad-hoc Logging	Schema-based Logging	Impact
Dashboard creation time	Hours	Minutes	90%+ reduction
Cross-service correlation	Manual	Automatic	Hours saved per incident
Compliance audit time	Days	Hours	80% reduction

Business impact of schema-based logging.

The storage savings come from better compression – consistent field names compress dramatically better than arbitrary strings. The query accuracy improvement comes from knowing *exactly* which field to search. The time savings compound: every query, every dashboard, every alert benefits from the consistency.

INFO

JSON (JavaScript Object Notation) is a format, not a schema. Two services can emit valid JSON (JavaScript Object Notation) logs with completely incompatible field names. Structured logging requires both: a machine-readable format *and* a shared vocabulary.

So how do you design a schema that actually sticks?

Designing Your Log Schema

Every schema decision you make in month one will constrain your queries for years. Get the fundamentals right before any service ships a log.

Schema Design Principles

Six rules that prevent the most common schema regrets:



#	Principle	Rationale	Good	Bad
1	Namespaced field names	Prevents collisions, enables discovery	<code>http.request.method</code> , <code>user.id</code>	<code>method</code> , <code>id</code> (ambiguous)
2	Consistent data types	Enables indexing and aggregation	<code>user.id</code> : always string	<code>user.id</code> : sometimes string, sometimes int
3	ISO 8601 timestamps	Timezone-aware, sortable, universal	<code>2024-01-15T14:30:00.123Z</code>	<code>01/15/2024 2:30 PM</code> , epoch ms
4	Flat over nested	Most log platforms query flat fields better	<code>http.request.headers.content_type</code>	<code>http: { request: { headers: {...} }}</code>
5	Enums for categories	Enables faceted search, prevents typos	<code>log.level</code> : error, warn, info	<code>level</code> : ERROR, Error, err
6	Units in field names	Prevents calculation errors	<code>http.response.time_ms</code>	<code>response_time</code> (seconds? ms?)

Core schema design principles.

The flat-over-nested rule surprises people. Dot notation like `http.request.method` looks nested but stores flat in Elasticsearch. Actual nested JSON (JavaScript Object Notation) objects require special nested field mappings, slow down queries, and complicate aggregations. Use dots in field names, not actual object nesting.

Adopting Elastic Common Schema (ECS)

Don't invent a schema. Elastic Common Schema (ECS (Elastic Common Schema)) <<https://www.elastic.co/docs/reference/ecs>> is the standard worth adopting.

ECS (Elastic Common Schema) provides namespaced fields for every common logging scenario: HTTP requests (`http.*`), errors (`error.*`), users (`user.*`), services (`service.*`), traces (`trace.*`), and events (`event.*`). The field names are already documented. The data types are already specified. Dashboards and detection rules exist that expect ECS (Elastic Common Schema) fields.



Here's a Pino¹ configuration that emits ECS (Elastic Common Schema)-compliant logs:

src/logging/ecs-logger.ts

```
1  import pino from 'pino'
2
3  const logger = pino({
4    formatters: {
5      level(label) {
6        return { 'log.level': label }
7      },
8    },
9    messageKey: 'message',
10   timestamp: () => `,"@timestamp":${new Date().toISOString()}`,
11   base: {
12     'service.name': process.env.SERVICE_NAME,
13     'service.version': process.env.SERVICE_VERSION,
14     'service.environment': process.env.NODE_ENV,
15     'service.node.name': process.env.HOSTNAME || process.env.POD_NAME,
16   },
17 })
18
19 export { logger }
```

ECS (Elastic Common Schema)-compliant Pino logger.

Usage produces logs that any ECS (Elastic Common Schema)-aware tool can parse:

src/handlers/orders.ts

```
1  logger.info({
2    'http.request.method': 'POST',
3    'http.request.path': '/api/orders',
4    'http.response.status_code': 201,
5    'user.id': 'usr_12345',
6    'event.category': ['web'],
7    'event.action': 'order_created',
8    'event.outcome': 'success',
9  }, 'Order created successfully')
```



ECS (Elastic Common Schema) request logging example.

The key ECS (Elastic Common Schema) namespaces you'll use most often:

Namespace	Purpose	Common Fields
service.*	Service identification	service.name, service.version, service.environment
http.*	HTTP request/response	http.request.method, http.response.status_code
error.*	Error details	error.type, error.message, error.stack_trace
user.*	User context	user.id, user.name, user.roles
trace.*	Distributed tracing	trace.id, span.id
event.*	Event categorization	event.action, event.outcome, event.category

Common ECS namespaces.

Custom Fields and Extensions

ECS (Elastic Common Schema) won't cover everything. Order IDs, customer tiers, feature flags – your domain has fields that no standard anticipates. The question isn't whether to add custom fields, but **how** to add them without creating the inconsistency problems you're trying to solve.

When you need custom fields, follow ECS (Elastic Common Schema) conventions: use a dedicated namespace, document it in your schema registry, and keep it minimal. Every custom field you add is a field your teammates need to learn.

src/logging/order-logger.ts

```

1  // Extend the base logger with order-specific fields
2  const orderLogger = logger.child({
3    'order.id': order.id,
4    'order.status': order.status,           // 'pending' | 'confirmed' | 'shipped'
5    'order.total_cents': order.totalCents,  // Units in field name

```



```
6     'order.item_count': order.items.length,  
7     'customer.tier': customer.tier,           // 'free' | 'premium' | 'enterprise'  
8   })  
9  
10  orderLogger.info({ 'event.action': 'order_confirmed' }, 'Order confirmed')
```

Custom namespace `order.*` for domain-specific fields. Note that amounts use cents (avoiding float precision issues) and the field name includes the unit.

⚠ WARNING

Custom namespaces should be rare. Before adding `order.shipping_method`, check if ECS (Elastic Common Schema) already has a field that fits. Custom fields increase cognitive load and reduce interoperability with pre-built dashboards and detection rules.

Correlation ID Implementation

A schema gets you consistent field names. But when a request fails, you need more than consistency – you need to see *every log from every service* that touched that request. That's where correlation IDs come in.

A single user action – placing an order, uploading a file, logging in – touches multiple services: API gateways, business logic services, payment processors, notification systems, databases. Without correlation IDs, each service's logs are islands. You know *something* failed, but stitching together the sequence of events across service boundaries requires manual timestamp correlation and guesswork.

Correlation IDs solve this by threading a common identifier through every log entry related to a single logical operation. Query by that ID, and you get the complete story – regardless of how many services participated.

Correlation ID Types

Not all correlation happens at the same scope. A single user session might span hundreds of requests, each request might fan out to dozens of service calls, and async operations might continue long after the original request completes. Different ID types address different correlation needs:



#	ID Type	Scope	Generated By	Lifespan	Use Case
1	<code>trace.id</code>	Distributed trace	Edge/first service	Single request tree	Debugging request flow
2	<code>span.id</code>	Single operation	Each service	One operation	Identifying service involvement
3	<code>request.id</code>	HTTP request	API gateway	Single HTTP request	Correlating gateway with downstream
4	<code>session.id</code>	User session	Auth system	Session duration	Tracking user activity across requests
5	<code>transaction.id</code>	Business operation	Domain service	Until completion	Tracking sagas and workflows
6	<code>causation.id</code>	Event chain	Message producer	One hop	Understanding event triggers

Correlation ID types and scopes.

The interface below shows the full correlation context. In practice, you won't use every field in every log – include what you'll actually query:

src/correlation/types.ts

```

1  // Correlation ID hierarchy
2  interface CorrelationContext {
3    // Request-level: entire user request across all services
4    requestId: string           // Generated at edge/API gateway
5
6    // Trace-level: distributed trace context (OpenTelemetry)
7    traceId: string            // Propagated via W3C Trace Context
8    spanId: string             // Current operation within trace
9    parentSpanId?: string      // Parent operation
10
11   // Session-level: user session across multiple requests

```



```

12     sessionId?: string      // From session cookie or JWT
13
14     // Business transaction: logical operation across requests
15     transactionId?: string  // e.g., order fulfillment saga
16
17     // Causation: what triggered this operation
18     causationId?: string    // ID of the event/message that caused this
19 }

```

Correlation context interface.

When all correlation IDs are present, a single log entry becomes a portal to multiple views of the same operation:

src/handlers/payment.ts

```

1  // Example log showing all correlation levels
2  const logEntry = {
3    '@timestamp': '2024-01-15T14:30:00.123Z',
4    'message': 'Payment authorized',
5
6    // All correlation IDs present
7    'trace.id': '4bf92f3577b34da6a3ce929d0e0e4736',
8    'span.id': '00f067aa0ba902b7',
9    'transaction.id': 'order-12345',           // Business transaction
10   'request.id': 'req_abc123',                // Edge-assigned request ID
11   'session.id': 'sess_xyz789',              // User session
12   'event.causation_id': 'order_created_evt', // What triggered this
13 }
14
15 // This single log entry enables:
16 // - trace.id query: all logs for this request across services
17 // - transaction.id query: entire order fulfillment saga
18 // - session.id query: everything this user did in their session
19 // - causation_id query: what event triggered this payment

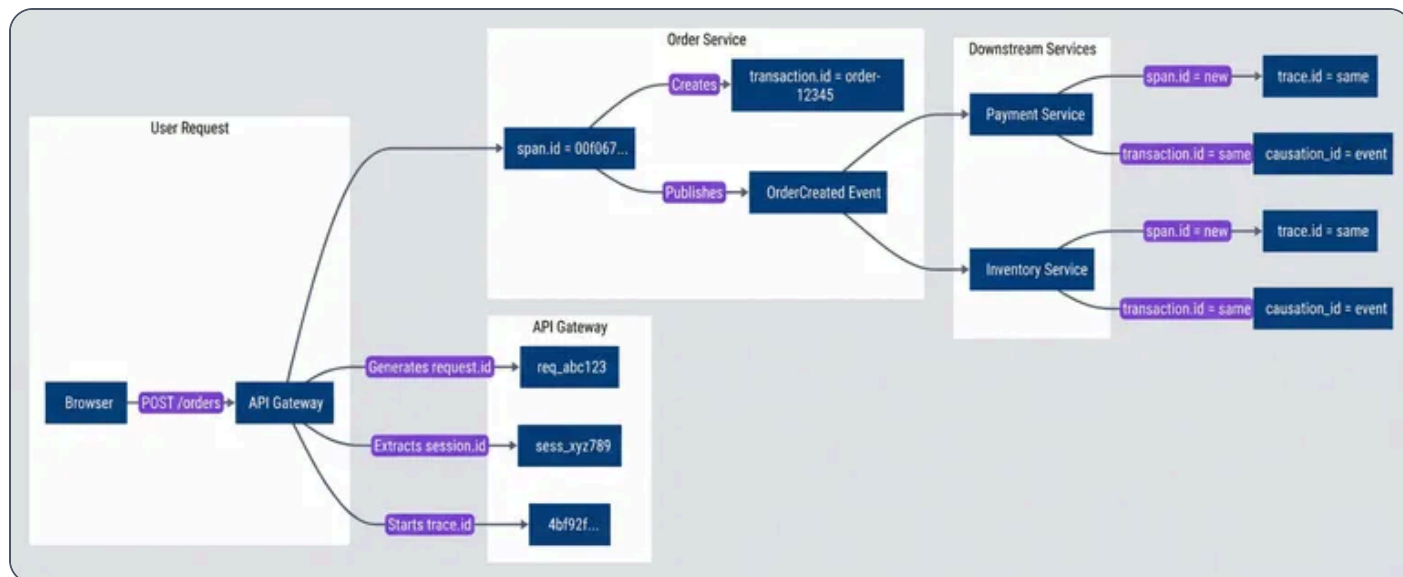
```

Log entry with correlation IDs.



How Correlation Flows Through Services

When a user places an order, the request flows through multiple services. Each service creates its own span but preserves the trace ID, allowing the entire flow to be reconstructed. Watch for two things in this diagram: trace IDs stay constant (that's how you query across services), while span IDs change at each hop (that's how you identify **which** service you're looking at):



Correlation ID flow across services.

The key insight: `trace.id` stays constant across all services for a single request tree. Query `trace.id:4bf92f*` and you see *every* log from *every* service involved in that order placement – API gateway, order service, payment service, inventory service, notification service.

INFO

The W3C Trace Context standard defines how to propagate trace and span IDs in HTTP headers via the `traceparent` header. Use it – it's supported by OpenTelemetry, Jaeger, Zipkin, and most APM (Application Performance Monitoring) tools. Don't invent your own trace header format.



Propagation Mechanisms

Correlation IDs are useless if they don't propagate. Every service-to-service call – HTTP, gRPC, message queue – must carry the correlation context forward.

HTTP Header Propagation

For synchronous HTTP calls, correlation IDs travel in headers. The W3C Trace Context standard defines

`traceparent` for trace/span IDs; use custom `x-*` headers for application-specific IDs:

src/correlation/headers.ts

```
1 // HTTP header propagation
2 const correlationHeaders = {
3   // W3C Trace Context (standard)
4   traceparent: '00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01',
5   tracestate: 'vendor=value',
6
7   // Custom headers for non-trace IDs
8   'x-request-id': 'req_abc123',
9   'x-session-id': 'sess_xyz789',
10  'x-transaction-id': 'order-12345',
11  'x-causation-id': 'order_created_evt',
12 }
```

Correlation HTTP headers.

The challenge: correlation context must be available to any code that logs or makes outbound calls, without explicitly passing it through every function. Node.js's `AsyncLocalStorage` solves this by maintaining context across async boundaries:

src/correlation/middleware.ts

```
1 // Express middleware for extraction and injection
2 import { AsyncLocalStorage } from 'async_hooks'
3
4 interface RequestContext {
5   traceId: string
```



```
6     spanId: string
7     requestId: string
8     sessionId?: string
9     transactionId?: string
10  }
11
12  const contextStorage = new AsyncLocalStorage<RequestContext>()
13
14  function correlationMiddleware(req: Request, res: Response, next: NextFunction) {
15    // Extract or generate IDs
16    const traceHeader = req.headers['traceparent'] as string
17    const { traceId, spanId } = parseTraceParent(traceHeader) || {
18      traceId: generateTraceId(),
19      spanId: generateSpanId()
20    }
21
22    const context: RequestContext = {
23      traceId,
24      spanId,
25      requestId: req.headers['x-request-id'] as string || generateRequestId(),
26      sessionId: req.headers['x-session-id'] as string,
27      transactionId: req.headers['x-transaction-id'] as string,
28    }
29
30    // Set response headers for debugging
31    res.setHeader('x-trace-id', traceId)
32    res.setHeader('x-request-id', context.requestId)
33
34    // Run request with context
35    contextStorage.run(context, () => next())
36  }
37
38  // Get current context from anywhere
39  function getCorrelationContext(): RequestContext | undefined {
40    return contextStorage.getStore()
41  }
```

AsyncLocalStorage correlation middleware.

The middleware above handles **extraction** – pulling correlation IDs from incoming requests. But that’s only half the story. You also need **injection** – adding correlation IDs to outbound requests. Here’s a fetch wrapper that does that automatically:



src/correlation/client.ts

```
1 // HTTP client that propagates context
2 async function fetchWithCorrelation(url: string, options: RequestInit = {}) {
3   const context = getCorrelationContext()
4
5   const headers = new Headers(options.headers)
6
7   if (context) {
8     // Generate new span ID for outbound call (same trace, new span)
9     headers.set('traceparent', formatTraceParent(context.traceId, generateSpanId()))
10    headers.set('x-request-id', context.requestId)
11    if (context.sessionId) headers.set('x-session-id', context.sessionId)
12    if (context.transactionId) headers.set('x-transaction-id', context.transactionId)
13  }
14
15  return fetch(url, { ...options, headers })
16 }
```

Correlation-aware HTTP client.

⚠ WARNING

Every HTTP client in your codebase must propagate correlation headers. A single direct `fetch()` call breaks the trace chain. Wrap your HTTP clients or use instrumentation libraries like OpenTelemetry's auto-instrumentation to ensure consistency.

Message Queue Correlation

Async messaging requires correlation context embedded in the message envelope itself, since HTTP headers don't exist. Messages also track *causation* – which previous message or request triggered them:

src/correlation/messaging.ts

```
1 // Message envelope carries correlation through queues
2 interface MessageEnvelope<T> {
```



```

3    messageId: string
4    timestamp: string
5    type: string
6    correlation: {
7      traceId: string
8      causationId: string      // What triggered this message
9      originRequestId?: string // Original HTTP request that started the chain
10   }
11   payload: T
12 }
13
14 // Producer: wrap payload with correlation from current context
15 async function publishEvent<T>(type: string, payload: T): Promise<void> {
16   const ctx = getCorrelationContext()
17   const envelope: MessageEnvelope<T> = {
18     messageId: generateMessageId(),
19     timestamp: new Date().toISOString(),
20     type,
21     correlation: {
22       traceId: ctx?.traceId || generateTraceId(),
23       causationId: ctx?.causationId || 'http_request',
24       originRequestId: ctx?.requestId,
25     },
26     payload,
27   }
28   await messageQueue.publish(type, envelope)
29 }
30
31 // Consumer: restore correlation before processing
32 async function handleMessage<T>(envelope: MessageEnvelope<T>): Promise<void> {
33   const context: RequestContext = {
34     traceId: envelope.correlation.traceId,
35     spanId: generateSpanId(),
36     causationId: envelope.messageId, // This message is now the cause of downstream
work
37   }
38
39   await contextStorage.run(context, () => processMessage(envelope.payload))
40 }

```

Message queue correlation pattern.



Correlation Anti-Patterns

I've seen all of these break correlation in production. Each one creates gaps in your trace data that make debugging harder:

Anti-Pattern	Why It Breaks	Solution
Generating new trace IDs per service	Logs can't be joined across services	Extract incoming <code>traceparent</code> ; generate only if missing
Logging outside <code>AsyncLocalStorage</code>	Logs lack correlation context	Run all request handlers within <code>contextStorage.run()</code>
Direct HTTP client usage	Outbound calls don't propagate headers	Use <code>fetchWithCorrelation()</code> or OpenTelemetry auto-instrumentation
Fire-and-forget async	Background work loses context	Capture context before spawning; restore in async handler
Batched message processing	All messages share one correlation	Process each message in its own <code>contextStorage.run()</code> scope

Correlation anti-patterns and fixes.

✓ SUCCESS

Test correlation by making a request, collecting logs from all services, and verifying they share the same `trace.id`. If any service's logs are missing from the trace, you have a propagation gap. Automated tests should verify this on every deployment.

Logger Implementation Patterns

Schema compliance requires enforcement, not just documentation. Developers under deadline pressure will log whatever's convenient — `console.log('user:', user)` is faster than constructing an ECS (Elastic Common Schema)-compliant object. The solution is making the right thing the easy thing: build loggers that enforce schema compliance by default, so developers get it right without extra effort.



Type-Safe Logger Factory

TypeScript's type system can catch schema violations at compile time. By defining interfaces for log field groups and requiring them in method signatures, you shift validation from runtime (where it's often skipped) to build time (where it can't be ignored). Here's the foundation – interfaces that map to ECS (Elastic Common Schema) field groups:

src/logging/structured-logger.ts

```
1  // Type definitions enforce ECS compliance at compile time
2  interface HttpLogFields {
3    'http.request.method': string
4    'http.request.path': string
5    'http.response.status_code': number
6    'http.response.time_ms': number
7  }
8
9  class StructuredLogger {
10   private baseFields: Record<string, unknown>
11
12   constructor(config: { serviceName: string; serviceVersion: string }) {
13     this.baseFields = {
14       'service.name': config.serviceName,
15       'service.version': config.serviceVersion,
16     }
17   }
18
19   private log(level: string, message: string, fields: Record<string, unknown>) {
20     const context = getCorrelationContext()
21     const entry = {
22       '@timestamp': new Date().toISOString(),
23       'log.level': level,
24       message,
25       ...this.baseFields,
26       ...(context && { 'trace.id': context.traceId, 'span.id': context.spanId }),
27       ...fields,
28     }
29     console.log(JSON.stringify(entry))
30   }
31
32   info(message: string, fields?: Record<string, unknown>) {
```



```
33     this.log('info', message, fields || {})
34   }
35
36   error(message: string, error: Error, fields?: Record<string, unknown>) {
37     this.log('error', message, {
38       'error.type': error.name,
39       'error.message': error.message,
40       'error.stack_trace': error.stack,
41       ...fields,
42     })
43   }
44
45   // Typed method: missing http.response.status_code = compile error
46   httpRequest(message: string, fields: HttpLogFields & Record<string, unknown>) {
47     this.log('info', message, { 'event.category': ['web'], ...fields })
48   }
49
50   // Child logger inherits base fields + adds new ones
51   child(fields: Record<string, unknown>): StructuredLogger {
52     const child = Object.create(this)
53     child.baseFields = { ...this.baseFields, ...fields }
54     return child
55   }
56 }
```

Type-safe structured logger.

The `child()` pattern is powerful for domain contexts – create a child logger when entering an operation, and all logs inherit the bound fields:

src/handlers/orders.ts

```
1  // Usage
2  const logger = new StructuredLogger({
3    serviceName: 'order-service',
4    serviceVersion: '1.2.3',
5  })
6
7  const orderLogger = logger.child({ 'order.id': 'ord_12345' })
8
9  orderLogger.httpRequest('Order created', {
```



```
10   'http.request.method': 'POST',
11   'http.request.path': '/api/orders',
12   'http.response.status_code': 201,
13   'http.response.time_ms': 45,
14   'event.action': 'order_created',
15   })
```

Child logger example.

Request-Scoped Logging

The child logger pattern works well for domain contexts, but every HTTP handler has the same boilerplate: extract correlation context, add request metadata, bind user info. Instead of repeating that everywhere, create a request-scoped logger in middleware that handlers can use directly:

src/middleware/logging.ts

```
1  // Middleware creates a pre-configured logger for each request
2  function loggingMiddleware(req: Request, res: Response, next: NextFunction) {
3    const context = getCorrelationContext()
4    const startTime = Date.now()
5
6    // Create logger with all request context pre-bound
7    req.log = logger.child({
8      'http.request.method': req.method,
9      'http.request.path': req.path,
10     'trace.id': context?.traceId,
11     'span.id': context?.spanId,
12     ...(req.user && { 'user.id': req.user.id }),
13   })
14
15   req.log.info('Request started', { 'event.action': 'http_request_started' })
16
17   res.on('finish', () => {
18     req.log.httpRequest('Request completed', {
19       'http.request.method': req.method,
20       'http.request.path': req.path,
21       'http.response.status_code': res.statusCode,
22       'http.response.time_ms': Date.now() - startTime,
23       'event.outcome': res.statusCode < 400 ? 'success' : 'failure',
```



```
24     })
25   })
26
27   next()
28 }
```

Request-scoped logging middleware.

Now handlers just call `req.log.info()` without worrying about context:

src/handlers/orders.ts

```
1  app.post('/api/orders', async (req, res) => {
2    req.log.info('Processing order', { 'order.item_count': req.body.items.length })
3
4    const order = await createOrder(req.body)
5    req.log.info('Order created', { 'order.id': order.id, 'event.action': 'order_created'
6      })
7    res.status(201).json(order)
8  })
```

Handler with a request logger.

i INFO

This pattern works with Express, Fastify, Koa, and Hapi. The key is attaching the logger early and capturing response details in a completion callback.

Sensitive Data Handling

You've standardized your schema and implemented correlation IDs. Now every service emits queryable, traceable logs. But there's a catch: structured logging makes data leaks **easier**.



When everything was unstructured text, finding sensitive data required reading logs manually. Now a single query like `user.email:*` returns every log containing email addresses. That's powerful for debugging – and catastrophic if logs are compromised.

The goal isn't eliminating sensitive data from logs entirely. Sometimes you need an email address to debug a user-specific issue. The goal is **deliberate** handling: know what's being logged, who can access it, how long it's retained, and whether it can be recovered if needed.

Redaction Strategies

Different data requires different treatment. Passwords should never appear – not even hashed. Email addresses might be partially masked for debugging while protecting the full value. Some fields need to be recoverable by security teams investigating fraud.

The redactor walks through log objects applying appropriate strategies based on field names. Pattern matching catches common sensitive fields; specific masking rules handle emails, phones, and card numbers:

src/logging/redactor.ts

```
1  class LogRedactor {
2    private sensitivePatterns = /password|secret|key|token|auth|credential/i
3
4    redact(obj: Record<string, unknown>): Record<string, unknown> {
5      const result: Record<string, unknown> = {}
6
7      for (const [key, value] of Object.entries(obj)) {
8        if (this.sensitivePatterns.test(key)) {
9          result[key] = '[REDACTED]'
10       } else if (typeof value === 'object' && value !== null) {
11         result[key] = this.redact(value as Record<string, unknown>)
12       } else if (typeof value === 'string') {
13         result[key] = this.maskValue(key, value)
14       } else {
15         result[key] = value
16       }
17     }
18     return result
19   }
20 }
```



```
21     private maskValue(key: string, value: string): string {
22         if (key.includes('email')) {
23             const [local, domain] = value.split('@')
24             return `${local[0]}***@${domain}`
25         }
26         if (key.includes('phone') || key.includes('card')) {
27             return value.replace(/\d(?:=\d{4})/g, '*')
28         }
29         return value
30     }
31 }
```

Log redaction example.

Field-Level Encryption

Some fields need to be recoverable – customer addresses for fraud investigations, transaction details for dispute resolution. For these, use AES-256-GCM encryption with key IDs that enable rotation:

src/logging/encryption.ts

```
1  class FieldEncryptor {
2      constructor(private key: Buffer, private keyId: string) {}
3
4      encrypt(value: string): EncryptedField {
5          const iv = randomBytes(16)
6          const cipher = createCipheriv('aes-256-gcm', this.key, iv)
7          const encrypted = cipher.update(value, 'utf8', 'base64') + cipher.final('base64')
8
9          return {
10             _encrypted: true,
11             keyId: this.keyId,
12             iv: iv.toString('base64'),
13             authTag: cipher.getAuthTag().toString('base64'),
14             ciphertext: encrypted,
15         }
16     }
17 }
18
19 // Usage: combine masking and encryption
```



```
20  logger.info('Customer registered', {  
21    'user.id': userId,  
22    'customer.email': redactor.maskEmail(email),      // Masked: debuggable  
23    'customer.address': encryptor.encrypt(address),   // Encrypted: recoverable  
24  })
```

Field encryption example.

⚠ DANGER

Passwords, API keys, tokens, and secrets should never appear in logs – not even hashed or encrypted. There's no legitimate debugging need for them. If they're appearing in logs, the logging location is wrong.

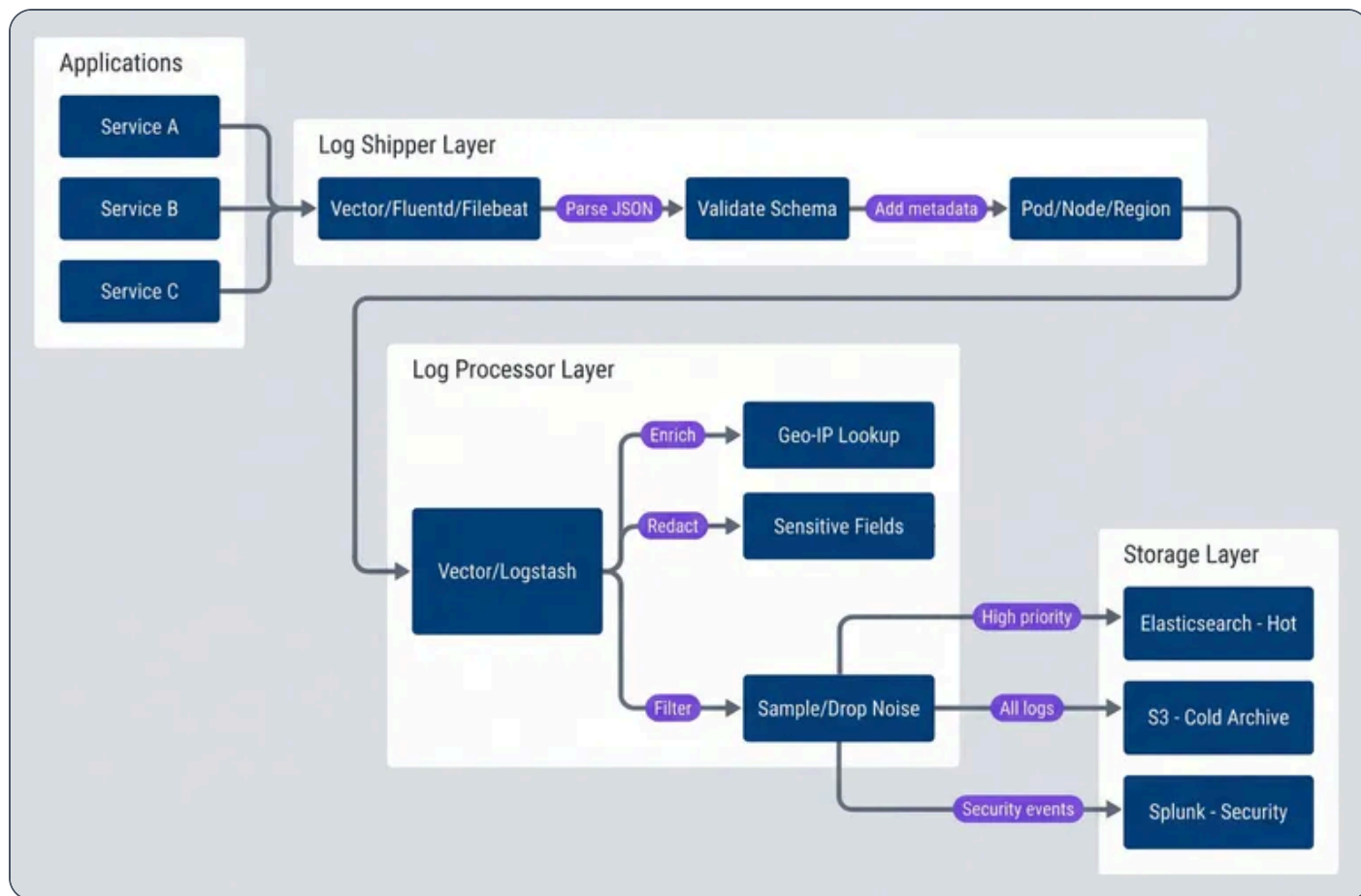
Collector-Side Processing

Application-level redaction catches most sensitive data, but defense in depth requires collector-side processing too. Log pipelines can catch data that slips through application controls, enrich logs with infrastructure context, and route logs to appropriate destinations based on content.

Log Pipeline Architecture

Modern log architectures have three tiers: shippers that collect logs from applications, processors that transform and route them, and storage that serves different query patterns.





Log pipeline architecture.

Noise Reduction Strategies

Not all logs deserve storage. Health checks fire every few seconds and rarely matter. Debug logs in production are noise by definition. Cache hits happen thousands of times per minute but add little debugging value. The goal is dropping low-value logs while keeping everything that might matter during an incident.

This framework helps decide how to handle different log patterns. The “Connection events” row uses aggregation – the same technique shown in the code example that follows:

Pattern	Action	Rationale
Health checks (/health, /ready)	Drop	High volume, rarely useful for debugging



Pattern	Action	Rationale
Metrics endpoint requests	Drop	Already captured in metrics, redundant
Debug logs in production	Drop	Should never reach production storage
Cache hits	Sample (1%)	Keep trend visibility, reduce volume
Connection events	Aggregate	Count per minute, not individual events
All errors	Keep	Every error is valuable for debugging
Authentication events	Keep	Security-relevant, audit requirements

Noise classification framework.

For high-volume events that still have value, aggregation preserves information while dramatically reducing storage. Instead of 1,000 individual connection logs per minute, emit one aggregated log:

src/logging/aggregation.ts

```

1  // Aggregation: count instead of individual logs
2  interface AggregatedLog {
3    '@timestamp': string
4    'log.level': 'info'
5    'message': string
6    'event.action': string
7    'aggregation.count': number
8    'aggregation.window_seconds': number
9  }
10
11 // Instead of 1000 "connection_opened" logs per minute:
12 const aggregatedConnectionLog: AggregatedLog = {
13   '@timestamp': '2024-01-15T14:31:00Z',
14   'log.level': 'info',

```



```
15     'message': 'Database connections aggregated',
16     'event.action': 'db_connection_aggregated',
17     'aggregation.count': 1247,
18     'aggregation.window_seconds': 60,
19   }
```

Aggregated log example.

⚠ WARNING

Be cautious with sampling and dropping. Start conservative – drop only the logs you’re *certain* have no debugging value. You can always drop more later, but you can’t recover logs you never stored.

Schema Governance and Evolution

A schema is only useful if everyone follows it. Without governance, you’ll have teams inventing their own field names, breaking consumers, and defeating the purpose of standardization. But governance can’t be so heavy that teams can’t add the fields they need.

Schema Registry Options

You need a central source of truth for field definitions – what fields exist, their types, who owns them, and whether they’re deprecated. Unfortunately, there’s no dominant open source “log schema registry” the way there’s a Confluent Schema Registry for Kafka. Your options:

Use your log platform's native schema management.

Elasticsearch has index templates and component templates that define field mappings. DataDog has facets. Splunk has field extractions. These work well if you're single-platform, but they're vendor-specific and don't help at log-emit time.



JSON Schema in your repo.

Define your schema as JSON Schema files, validate in CI (we'll cover this in the Testing section), and generate documentation from the schema. This is lightweight and works across platforms.



Build a lightweight registry service.

For larger organizations, a simple service that stores field definitions with ownership and versioning metadata. This does not need to be complex-a YAML file in a shared repo with a PR review process often suffices.



The key insight: the *format* of your registry matters less than *having one at all* and *enforcing it in CI*. A YAML file that every service validates against beats a sophisticated registry that nobody uses.

INFO

If you're using ECS (Elastic Common Schema), you already have a schema – the ECS (Elastic Common Schema) field reference <<https://www.elastic.co/guide/en/ecs/current/ecs-field-reference.html>> **is** your registry for standard fields. You only need to document custom fields you've added.

Schema Evolution Process

Schemas change – new features need new fields, old patterns get replaced. The key is managing change without breaking consumers. Here's the evolution process that's worked for me:

Phase	Duration	Action
Proposal	1 week	RFC for new field, review by platform team
Addition	Immediate	Add field as optional, update registry
Adoption	2-4 weeks	Teams start using new field



Phase	Duration	Action
Promotion	After adoption	Consider making field required if universal
Deprecation Notice	1 month	Mark field deprecated, log warnings
Migration Period	3 months	Teams migrate to replacement field
Removal	After migration	Remove field from registry, update parsers

Schema evolution timeline.

✓ SUCCESS

New fields must be optional. Parsers must ignore unknown fields. Type changes require a new field name. These rules keep old logs queryable and prevent producer/consumer version coupling.

Testing and Validating Your Logging

Everything we've covered – schemas, correlation, redaction – needs automated verification. Manual code review doesn't catch the developer who logs `userId` instead of `user.id`. CI tests do.

You need two things: tests that fail when logs violate the schema, and production monitoring that catches correlation gaps.

Schema Validation in CI

Use JSON (JavaScript Object Notation) Schema and Ajv to validate log output in unit tests. The pattern: capture log output, parse it, validate against your schema:



src/logging/__tests__/schema.spec.ts

```

1  import { describe, it, expect, beforeEach, vi } from 'vitest'
2  import Ajv from 'ajv'
3  import addFormats from 'ajv-formats'
4
5  const ecsLogSchema = {
6    type: 'object',
7    required: ['@timestamp', 'log.level', 'message', 'service.name'],
8    properties: { '@timestamp': { type: 'string', format: 'date-time' } },
9    additionalProperties: true,
10 }
11
12 describe('Structured Logger Schema Compliance', () => {
13   let logOutput: string[] = []
14   const validate = new Ajv({ allErrors: true
15   }).addFormats(addFormats).compile(ecsLogSchema)
16
17   beforeEach(() => {
18     logOutput = []
19     vi.spyOn(console, 'log').mockImplementation((msg: string) => logOutput.push(msg))
20   })
21
22   it('emits valid ECS-compliant logs', () => {
23     logger.info('Test message', { 'user.id': 'usr_123' })
24     const entry = JSON.parse(logOutput[0])
25
26     expect(validate(entry)).toBe(true)
27     expect(entry['@timestamp']).toMatch(/^d{4}-d{2}-d{2}T/)
28   })
29
30   it('includes trace context when available', () => {
31     logger.info('With trace', {
32       'trace.id': '4bf92f3577b34da6a3ce929d0e0e4736',
33       'span.id': '00f067aa0ba902b7',
34     })
35     const entry = JSON.parse(logOutput[0])
36
37     expect(entry['trace.id']).toMatch(/^[a-f0-9]{32}$/)
38     expect(entry['span.id']).toMatch(/^[a-f0-9]{16}$/)
39   })
40 })

```



Ajv schema validation tests.

Correlation ID Propagation Tests

Correlation is only useful if it propagates correctly. Test that trace IDs survive HTTP boundaries – both inbound extraction and outbound propagation:

src/correlation/__tests__/propagation.spec.ts

```
1  describe('Correlation ID Propagation', () => {
2    it('extracts trace context from W3C traceparent header', async () => {
3      const req = createMockRequest({
4        headers: { traceparent: '00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01'
5      },
6    })
7    await new Promise<void>(resolve => {
8      correlationMiddleware(req, createMockResponse(), () => {
9        expect(getCorrelationContext()?.traceId).toBe('4bf92f3577b34da6a3ce929d0e0e4736')
10      resolve()
11    })
12  })
13 })
14
15 it('generates IDs when headers are missing', async () => {
16   const req = createMockRequest({ headers: {} })
17
18   await new Promise<void>(resolve => {
19     correlationMiddleware(req, createMockResponse(), () => {
20       expect(getCorrelationContext()?.traceId).toMatch(/^[a-f0-9]{32}$/)
21       resolve()
22     })
23   })
24 })
25
26 it('propagates context to outbound requests', async () => {
27   const mockFetch = vi.fn().mockResolvedValue(new Response('{}'))
28   const testContext = { traceId: '4bf92f3577b34da6a', spanId: '00f067aa', requestId:
29     'req_123' }
```



```
30     await contextStorage.run(testContext, () =>
31       fetchWithCorrelation('https://api.example.com', {}, mockFetch)
32     )
33
34     expect(mockFetch.mock.calls[0]
35       [1].headers.get('traceparent')).toContain(testContext.traceId)
36   })
37 }
```

Correlation propagation tests.

Log Quality Metrics

In production, monitor log quality the same way you monitor application health. Three Prometheus counters give you visibility into schema compliance and correlation coverage:

- 1** `log_schema_validation_total{result, service}`
Counts valid vs. invalid log entries
- 2** `log_schema_violations_total{field, violation_type, service}`
Tracks which fields fail and why
- 3** `log_missing_correlation_total{service, log_location}`
Flags logs without trace IDs

Hook these counters into your logger's output path to track every log entry. Alert when violation rates spike or correlation coverage drops below your target (e.g., 99% of logs should have trace IDs).



 INFO

Set a correlation coverage target (e.g., 99% of logs must have trace IDs) and alert when it drops. Missing correlation IDs usually indicate logging from outside the request context – background jobs, startup code, or improperly instrumented libraries.

Conclusion

Structured logging at scale requires discipline across three dimensions: schema consistency, correlation propagation, and noise management.

Adopt ECS (Elastic Common Schema) rather than inventing a schema – it handles most use cases and enables cross-organization tooling compatibility. Implement correlation IDs at every boundary: HTTP headers for synchronous calls, message envelopes for async, and AsyncLocalStorage for automatic propagation within services. Redact sensitive data at the source, not the sink – assume logs will be accessed by anyone with read permissions.

Use collector-side processing to filter noise before storage: drop health checks and debug logs, sample high-volume events, aggregate repetitive patterns. Establish schema governance early; field naming decisions made in month one will constrain querying for years.

The next time it's 3 AM and you're chasing a bug across twelve services, you'll type one query instead of five – and you won't miss the inventory service.

-
- ❶ Pino is an extremely fast, low-overhead, and lightweight logger for Node.js applications, designed to minimize performance impact through asynchronous logging. It produces structured JSON (JavaScript Object Notation) logs by default, making it ideal for production environments, observability platforms, and log aggregation tools. ↩



Copyright © 2022 Webstack Builders, Inc.

The text, diagrams, and images in this work are licensed under CC BY-NC 4.0

All code samples in this article are licensed under the MIT License. Feel free to use, modify, and distribute them in any project.

<https://www.webstackbuilders.com/articles/structured-logging-correlation-ids-log-schema-design>

